

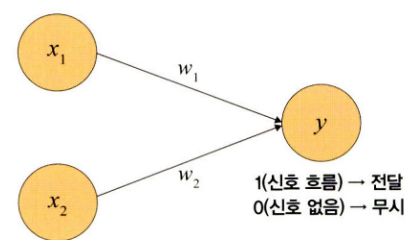
4.1 인공 신경망의 한계와 딥러닝 출현

✓ 인공 신경망의 구조

- 오늘날 인공 신경망에서 이용하는 구조(입력층, 출력층, 가중치로 구성된 구조) ⇒ 퍼셉트론

퍼셉트론

- 프랭크 로젠블라트가 1957년에 고안한 선형분류기
- 신경망(딥러닝)의 기원이 되는 알고리즘
- 다수의 신호(흐름이 있는)를 입력으로 받아 하나의 신호를 출력
- 신호를 입력으로 받아 '흐른다/안흐른다(1/0)'는 정보를 앞으로 전달하는 원리로 작동



논리 게이트

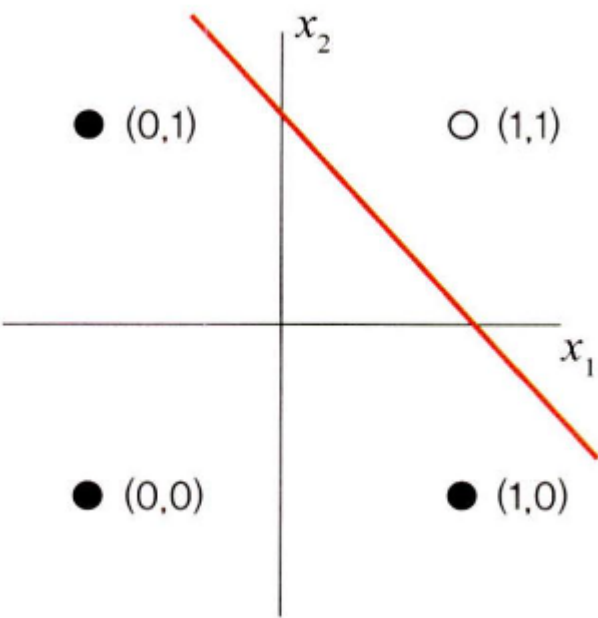
- 입력이 두 개(x_1, x_2) 있다고 할 때 컴퓨터가 논리적으로 인식하는 방식

1) AND 게이트

- 모든 입력이 '1'일 때 작동
- 입력 중 어떤 하나라도 '0'을 갖는다면 작동을 멈춤

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

- 데이터 분류(검은색 점과 흰색 점) 표현

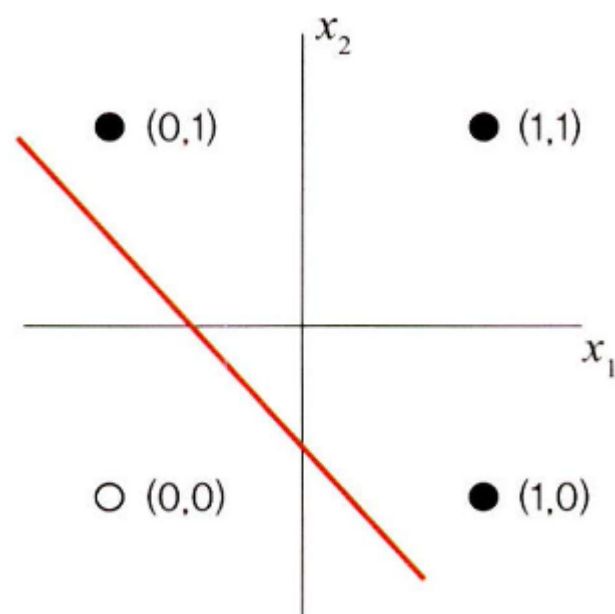


2) OR 게이트

- 입력에서 둘 중 하나만 '1'이거나 둘 다 '1'일 때 작동
- 입력 모두가 '0'을 갖는 경우를 제외한 나머지가 모두 '1'값을 가짐

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

- 데이터 분류(검은색 점과 흰색 점) 표현

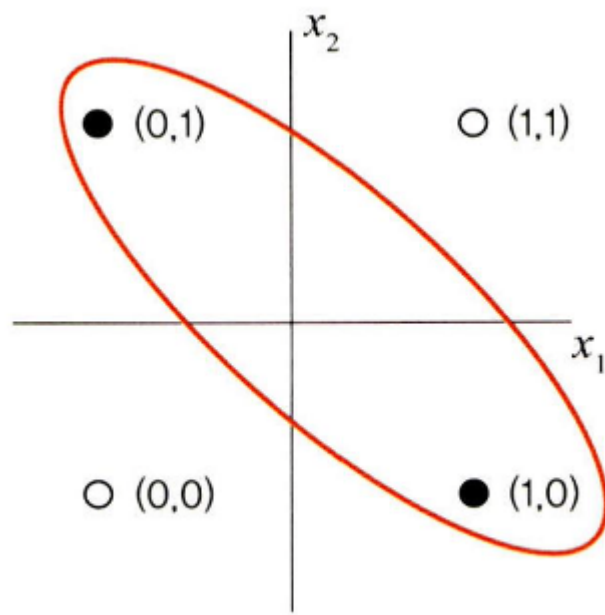


3) XOR 게이트

- 배타적 논리합
- 입력 두 개 중 한 개만 '1'일 때 작동하는 논리 연산

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

- XOR 게이트에서는 데이터가 비선형적으로 분리



⇒ 제대로 된 분류가 어려움

⇒ 퍼셉트론에서는 AND, OR 연산에 대해서는 학습이 가능하지만 XOR에 대해서는 학습이 불가능

⇒ 입력층과 출력층 사이에 하나 이상의 중간층(은닉층)을 둬

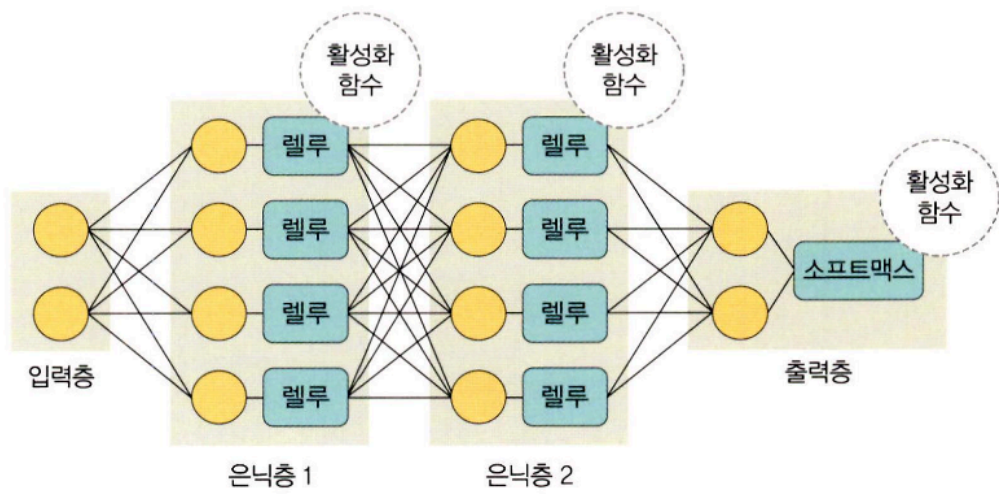
⇒ 비선형적으로 분리되는 데이터에 대해서도 학습이 가능하도록 다층 퍼셉트론을 고안

- **심층 신경망[딥러닝]:** 입력층과 출력층 사이에 은닉층이 여러 개 있는 신경망

4.2 딥러닝 구조

✓ 딥러닝 용어

✓ 딥러닝 구조, 구성 요소



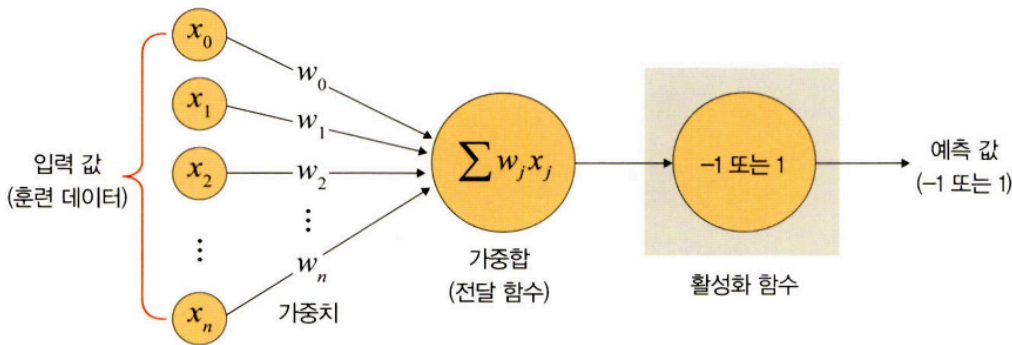
- 입력층, 출력층, 두 개 이상의 은닉층으로 구성
- 입력 신호를 전달하기 위한 다양한 함수 사용

구분	구성 요소	설명
층	입력층(input layer)	데이터를 받아들이는 층
	은닉층(hidden layer)	모든 입력 노드부터 입력 값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
	출력층(output layer)	신경망의 최종 결괏값이 포함된 층

구분	구성 요소	설명
가중치(weight)		노드와 노드 간 연결 강도
바이어스(bias)		가중합에 더해 주는 상수로, 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할을 함
가중합(weighted sum), 전달 함수		가중치와 신호의 곱을 합한 것
함수	활성화 함수(activation function)	신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
	손실 함수(loss function)	가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

✓ 가중치

: 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소

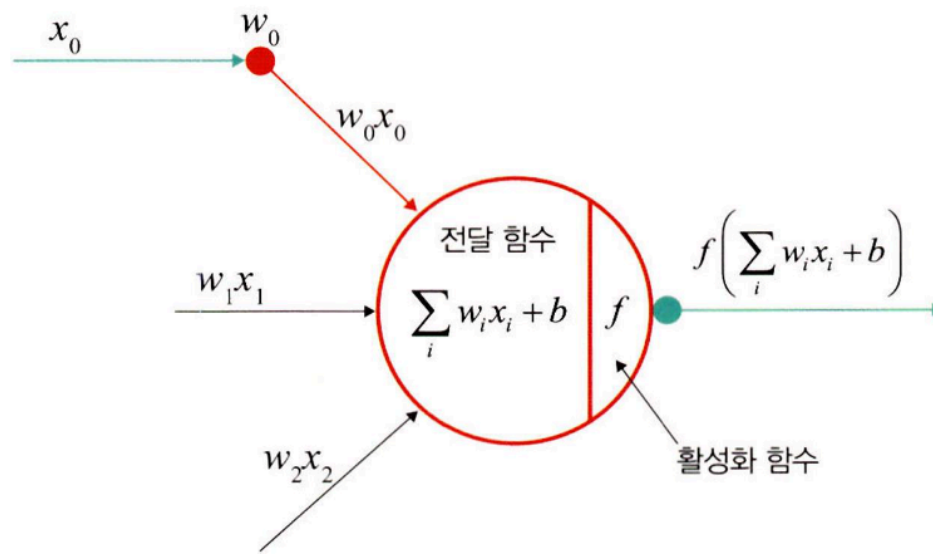


- w1 값이 0 혹은 0에 가까운 값이면 아무리 큰 값이어도 x1*w1의 값은 0이거나 0에 가까운 값이 됨

✓ 가중합/전달 함수

: 각 노드에서 들어오는 신호에 가중치를 곱한 값을 모두 더한 합계

- 각 노드에서 들어오는 신호에 가중치를 곱해서 다음 노드로 전달
- 노드의 가중합이 계산되면 이 값을 활성화 함수로 보내기 때문에 **전달 함수**라고도 함



- 가중합 구하는 공식

$$\sum_i w_i x_i + b$$

(w : 가중치, b : 바이어스)

✓ 활성화 함수

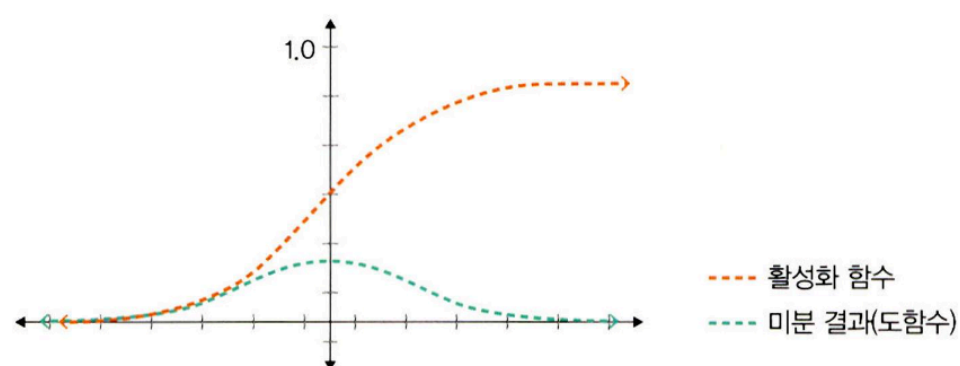
: 전달 함수에서 전달받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수
(비선형 함수: 직선으로 표현할 수 없는 데이터 사이의 관계를 표현하는 함수)

1) 시그모이드 함수

- 선형 함수의 결과를 0~1 사이에서 비선형 형태로 변형해줌
- 로지스틱 회귀와 같은 분류 문제를 확률적으로 표현하는 데 사용
- 과거에는 인기가 많았으나, 딥러닝 모델의 깊이가 깊어지면 기울기가 사라지는 '기울기 소멸 문제'가 발생하여 잘 사용하지 않음
- 수식

$$f(x) = \frac{1}{1 + e^{-x}}$$

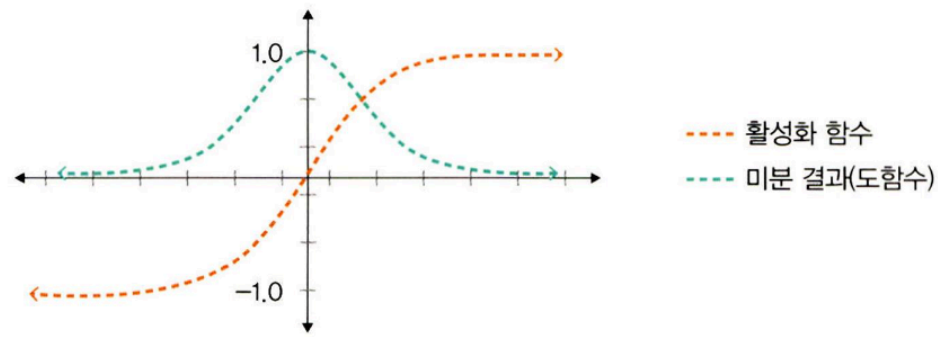
- 시그모이드 활성화 함수와 미분 결과



2) 하이퍼볼릭 탄젠트 함수

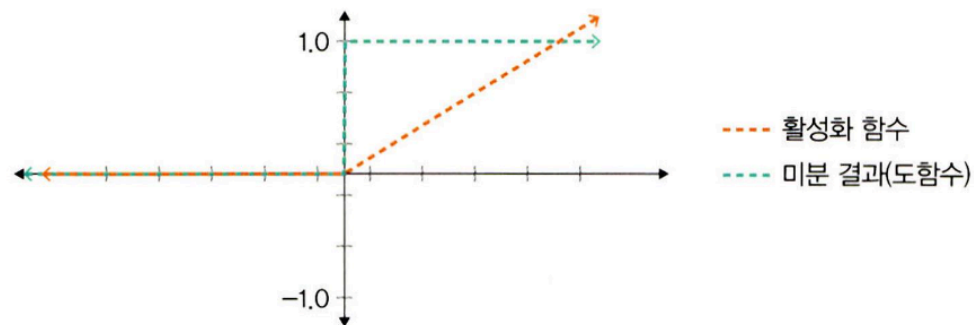
- 선형 함수의 결과를 -1 ~ 1 사이에서 비선형 형태로 변형해줌
- 시그모이드에서 결괏값의 평균이 0이 아닌 양수로 편향되는 문제를 해결하는 데 사용됨

- 기울기 소멸 문제는 여전히 발생
- **하이퍼볼릭 탄젠트 활성화 함수와 미분 결과**



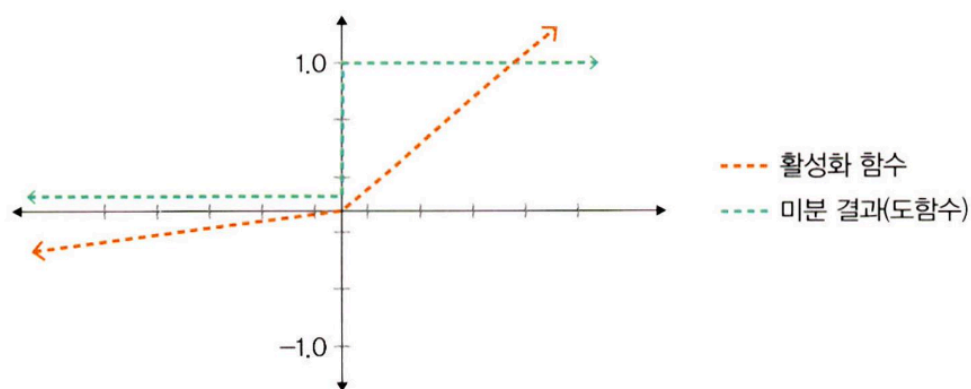
3) 렐루 함수

- 입력(x)이 음수일 때는 0을 출력, 양수일 때는 x를 출력
- 경사 하강법에 영향을 주지 않아 학습 속도가 빠르고, 기울기 소멸 문제가 발생하지 않음
- 일반적으로 은닉층에서 사용
- 하이퍼볼릭 탄젠트 함수 대비 학습 속도가 6배 빠름
- 음수 값을 입력받으면 항상 0을 출력하기 때문에 학습 능력이 감소
⇒ 리키 렐루 함수 등을 사용
- **렐루 활성화 함수와 미분 결과**



4) 리키 렐루 함수

- 입력 값이 음수이면 0이 아닌 0.001처럼 매우 작은 수를 반환
- 입력 값이 수렴하는 구간이 제거되어 렐루 함수를 사용할 때 생기는 문제가 해결
- **리키 렐루 활성화 함수와 미분 결과**



5) 소프트맥스 함수

- 입력 값을 0~1 사이에 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 함
- 딥러닝에서 출력 노드의 활성화 함수로 많이 사용됨
- 수식

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

- $\exp(x)$: 지수 함수
 - n : 출력층의 뉴런 개수
 - y_k : 그중 k 번째 출력
- ⇒ 소프트맥스 함수의 분자: 입력 신호 a_k 의 지수 함수로 구성
- 소프트맥스 함수의 분모: 모든 입력 신호의 지수 함수 합으로 구성

렐루 함수, 소프트맥스 함수 구현

```
class Net(torch.nn.Module):
    def __init__(self, n_feature, n_hidden, n_output):
        super(Net, self).__init__()
        self.hidden = torch.nn.Linear(n_feature, n_hidden) # 은닉층
        self.relu = torch.nn.ReLU(inplace=True)
        self.out = torch.nn.Linear(n_hidden, n_output) # 출력층
        self.softmax = torch.nn.Softmax(dim=n_output)
    def forward(self, x):
        x = self.hidden(x)
        x = self.relu(x) # 은닉층을 위한 렐루 활성화 함수
        x = self.out(x)
        x = self.softmax(x) # 출력층을 위한 소프트맥스 활성화 함수
        return x
```

✓ 손실 함수

• 경사 하강법

: 학습률(learning rate)과 손실 함수의 순간 기울기를 이용하여 가중치를 업데이트하는 방법

⇒ 미분의 기울기를 이용하여 오차를 비교하고 최소화하는 방향으로 이동시키는 방법

⇒ 이때 오차를 구하는 방법이 **손실 함수**

• 손실 함수

: 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표

- 값이 클수록 많이 틀렸다는 의미
- 값이 0에 가까울수록 완벽하게 추정할 수 있다는 의미

1) 평균 제곱 오차(MSE)

- 실제 값과 예측 값의 차이(error)를 제곱하여 평균을 낸 것
- 실제 값과 예측 값의 차이가 클수록 평균 제곱 오차의 값이 커짐
 - ⇒ MSE가 작을수록 예측력이 좋다는 의미
- 회귀에서 손실 함수로 주로 사용됨
- 수식

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$$

• 파이토치에서의 사용

```
import torch
```

```
loss_fn = torch.nn.MSELoss(reduction = 'sum')
y_pred = model(x)
loss = loss_fn(y_pred, y)
```

2) 크로스 엔트로피 오차(CEE)

- 분류 문제에서 원-핫 인코딩을 했을 때만 사용할 수 있는 오차 계산법
- 일반적으로 분류 문제에서는 데이터의 출력을 0과 1로 구문하기 위해 시그모이드 함수를 사용
 - ⇒ 시그모이드 함수에 포함된 자연 상수 e 때문에 평균 제곱 오차를 적용하면 매끄럽지 못한 그래프(울퉁불퉁한 그래프)가 출력
 - ⇒ 크로스 엔트로피 손실 함수 사용
- CEE를 적용할 경우, 경사 하강법 과정에서 학습이 지역 최소점에서 멈출 수 있음
 - ⇒ 이를 방지하기 위해 자연 상수 e 에 반대되는 자연 로그를 모델의 출력 값에 취함
- 수식

$$CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$$

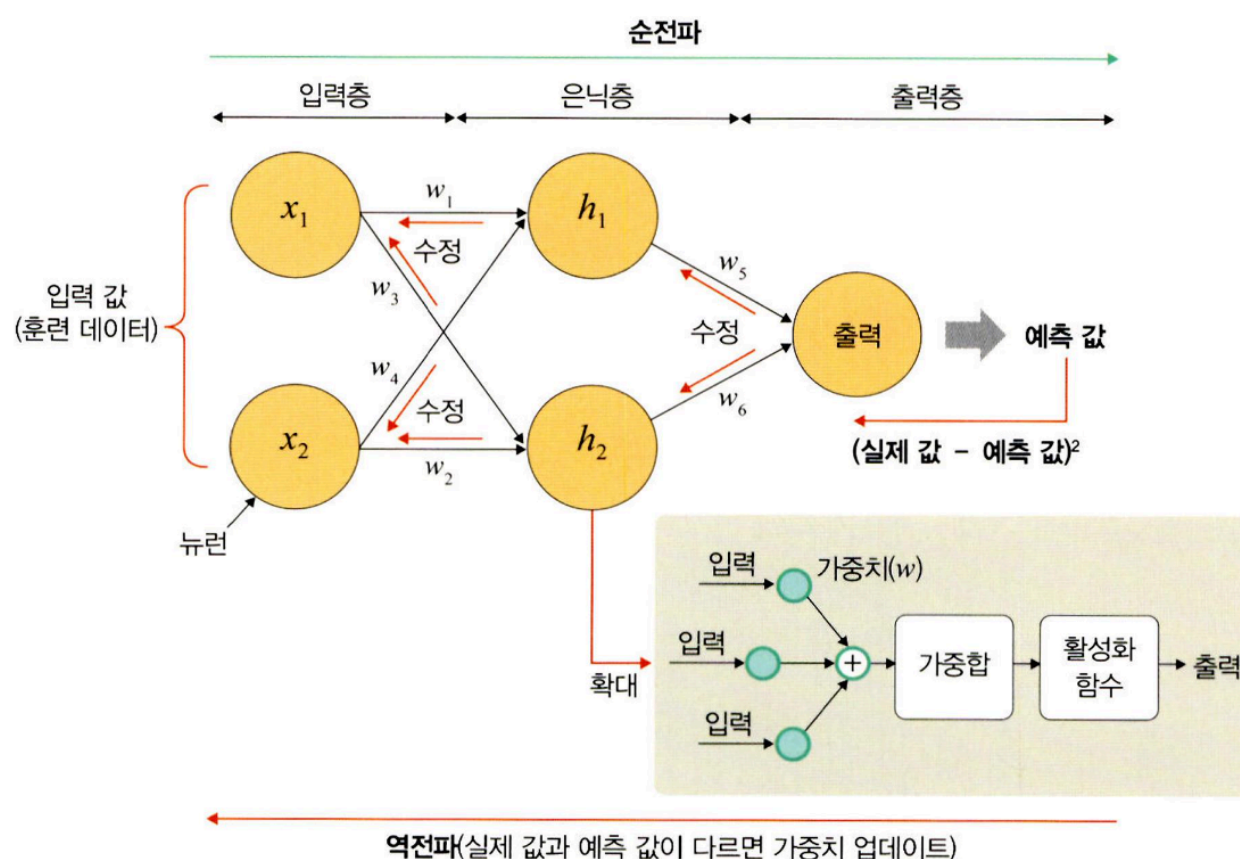
$\hat{y}_i$: 신경망의 출력(신경망이 추정한 값)
 y_i : 정답 레이블
 i : 데이터의 차원 개수

- 파이토치에서의 사용

```
loss = nn.CrossEntropyLoss()
input = torch.randn(5, 6, requires_grad = True)
# torch.randn는 평균이 0이고 표준편차가 1인 가우시안 정규분포를 이용하여 숫자 생성
target = torch.empty(3, dtype = torch.long).random_(5)
# torch.empty는 dtype torch.float32의 랜덤한 값으로 채워진 텐서를 반환
output = loss(input, target)
output.backward()
```

✓ 딥러닝 학습

- 딥러닝 학습은 크게 순전파와 역전파라는 두 단계로 진행



순전파

- 네트워크에 훈련 데이터가 들어올 때 발생
- 데이터를 기반으로 예측 값을 계산하기 위해 전체 신경망을 교차해 지나감
⇒ 모든 뉴런이 이전 층의 뉴런에서 수신한 정보에 변환(가중합&활성화 함수)을 적용하여 다음 층(은닉층)의 뉴런으로 전송하는 방식
- 입력 데이터가 신경망을 통해 흘러가면서 예측값을 만드는 과정

손실 함수

- "예측값"과 "실제값"의 차이를 계산
- 이때 손실 함수 비용은 0이 이상적

역전파

- 예측값이 틀렸다면, **신경망이 스스로 공부하면서 개선하는 과정**
- 출력층에서 시작된 손실 비용은 은닉층의 모든 뉴런으로 전파되지만, 은닉층의 뉴런은 각 뉴런이 원래 출력에 기여한 상대적 기여도(가중치)에 따라 값이 달라짐
- 예측 값과 실제 값 차이를 각 뉴런의 가중치로 미분한 후 기존 가중치 값에서 뺌

(순전파)

1. 네트워크를 통해 입력 데이터를 전달
2. 데이터가 모든 층을 통과하고 모든 뉴런이 계산을 완료하면 그 예측 값은 최종 층(출력층)에 도달

(손실함수)

3. 손실 함수로 네트워크의 예측 값과 실제 값의 차이(손실, 오차)를 추정

(역전파)

4. 손실 함수 비용이 0에 가깝도록 하기 위해 모델이 훈련을 반복하면서 가중치를 조정
5. 이 과정을 출력층→은닉층→입력층 순서로 모든 뉴런에 대해 진행하여 계산된 각 뉴런 결과를 또다시 순전파의 가중치 값으로 사용

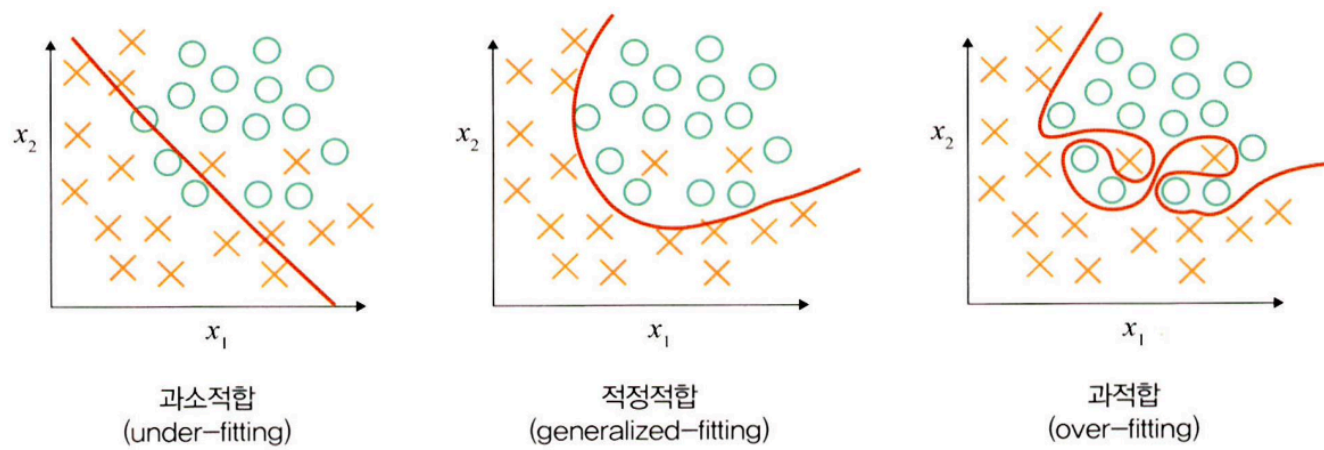
✓ 딥러닝의 문제점과 해결 방안

딥러닝의 은닉층

- 딥러닝의 핵심: 활성화 함수가 적용된 여러 은닉층을 결합하여 비선형 영역을 표현하는 것
- 은닉층의 개수가 많을수록 데이터 분류가 잘됨
- 은닉층이 많을 때 문제가 발생하기도 함
⇒ 과적합 문제, 기울기 소멸 문제, 성능 하락 문제

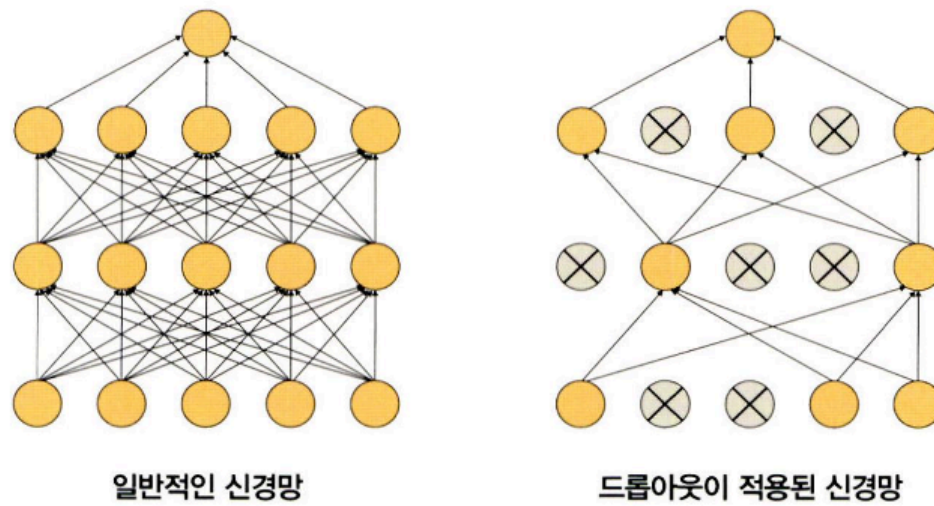
1. 과적합 문제

- 훈련 데이터를 과하게 학습해서 발생
 - 일반적으로 훈련 데이터는 실제 데이터의 일부분임.
 - 훈련 데이터를 과하게 학습했기 때문에 예측 값과 실제 값 차이인 오차가 감소하지만, 검증 데이터에 대해서는 오차가 증가
- 과적합: 훈련 데이터에 대해 과하게 학습하여 실제 데이터에 대한 오차가 증가하는 현상



해결 방안: 드롭아웃

- 신경망 모델이 과적합되는 것을 피하기 위한 방법
- 학습 과정 중 임의로 일부 노드들을 학습에서 제외시킴



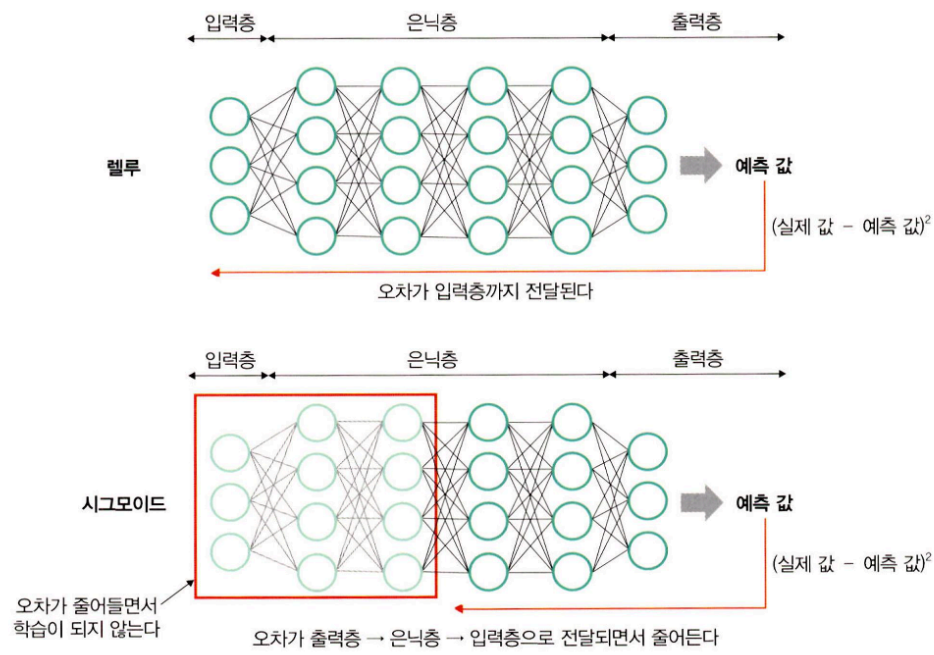
- 파이토치에서 드롭아웃 구현

```
class DropoutModel(torch.nn.Module):
    def __init__(self):
        super(DropoutModel, self).__init__()
        self.layer1 = torch.nn.Linear(784, 1200)
        self.dropout1 = torch.nn.Dropout(0, 5) # 50%의 노드를 무작위로 선택하여 사용하지 않겠다는 의미
        self.layer2 = torch.nn.Linear(1200, 1200)
        self.dropout2 = torch.nn.Dropout(0, 5)
        self.layer3 = torch.nn.Linear(1200, 10)

    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = self.dropout1(x)
        x = F.relu(self, layer2(x))
        x = self.dropout2(x)
        return self.layer3(x)
```

2. 기울기 소멸 문제

- 은닉층이 많은 신경망에서 주로 발생
- 출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상
- 기울기가 소멸되기 때문에 학습되는 양이 0에 가까워져 학습이 더디게 진행되다 오차를 더 줄이지 못하고 그 상태로 수렴하는 현상

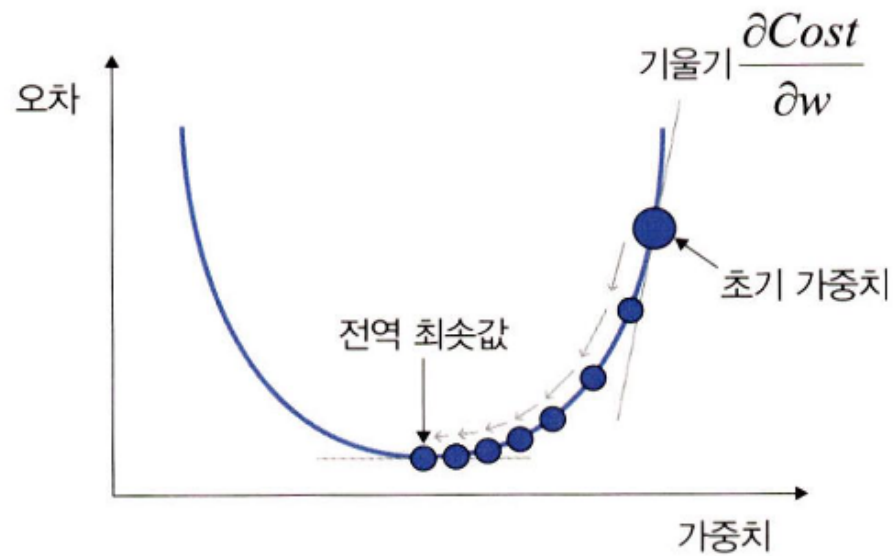


해결 방안

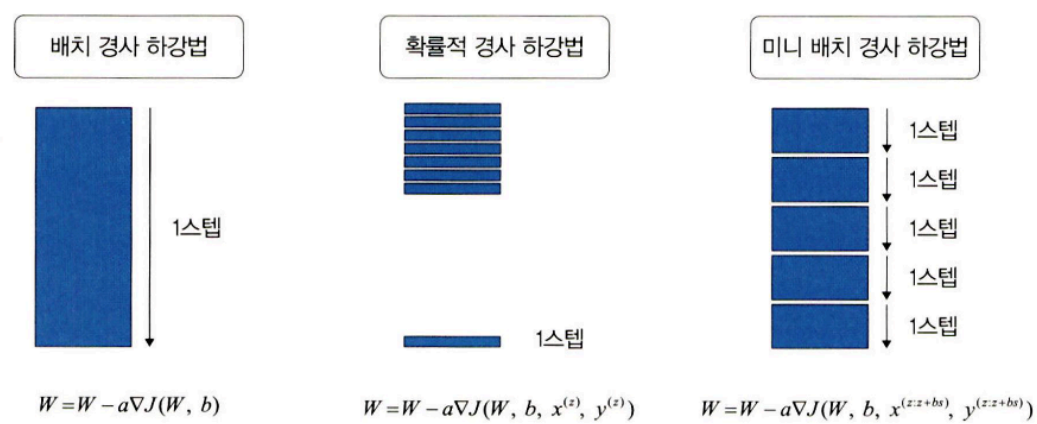
- 시그모이드 함수
- 하이퍼볼릭 탄젠트 함수
- 렐루 활성화 함수

3. 성능이 나빠지는 문제

- 경사 하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정을 반복
⇒ 성능이 나빠지는 문제 발생



해결 방안



1. 배치 경사 하강법

- 전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산하여 모델의 파라미터를 업데이트한 방법
- 전체 훈련 데이터셋에 대하여 가중치를 편미분하는 방법
- 수식

손실 함수의 값을 최소화하기 위해 기울기(∇) 이용

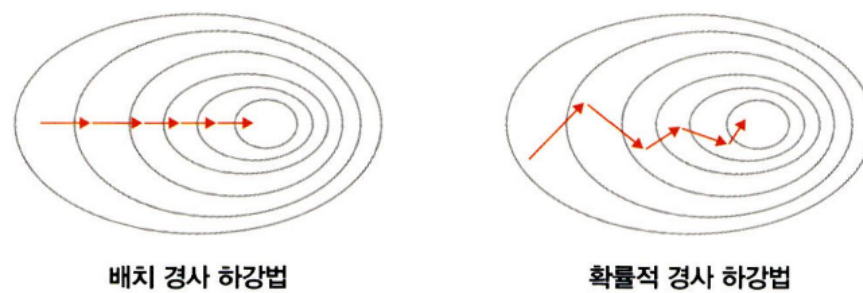
$$W = W - a \nabla J(W, b)$$

(a : 학습률, J : 손실 함수)

- 한 스텝에 모든 훈련 데이터셋을 사용
⇒ 학습이 오래 걸리는 단점
⇒ 이런 단점을 개선한 방법이 확률적 경사 하강법

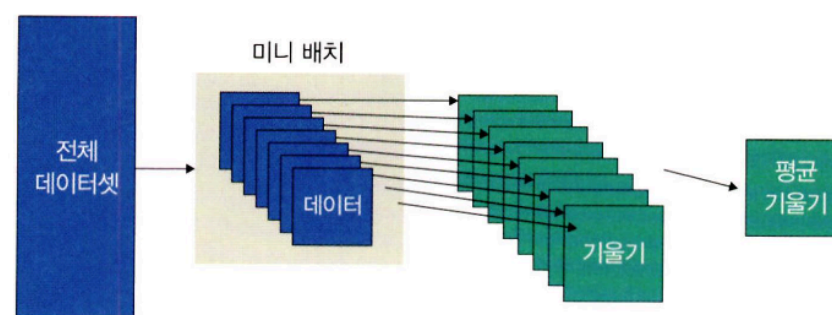
2. 확률적 경사 하강법

- 임의로 선택한 데이터에 대해 기울기를 계산하는 방법
- 적은 데이터 사용 ⇒ 빠른 계산
- 파라미터 변경 폭이 불안정하고, 때로는 배치 경사 하강법보다 정확도가 낮을 수 있음

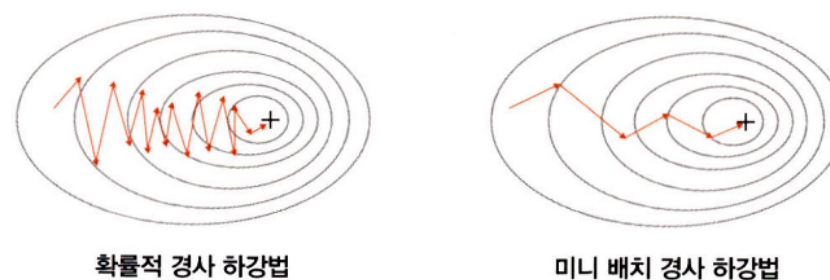


3. 미니 배치 경사 하강법

- 전체 데이터셋을 미니 배치 여러 개로 나누고, 미니 배치 한 개마다 기울기를 구한 후 그것의 평균 기울기를 이용하여 모델을 업데이트 해서 학습하는 방법



- 전체 데이터를 계산하는 것보다 빠름
- 파라미터 변경 폭이 확률적 경사 하강법보다 안정적
- 가장 많이 사용



- 파이토치에서의 구현

```
class CustomDataset(Dataset):
    def __init__(self):
        self.x_data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
        self.y_data = [[12], [18], [11]]
    def __len__(self):
        return len(self.x_data)
    def __getitem__(self, idx):
```

```

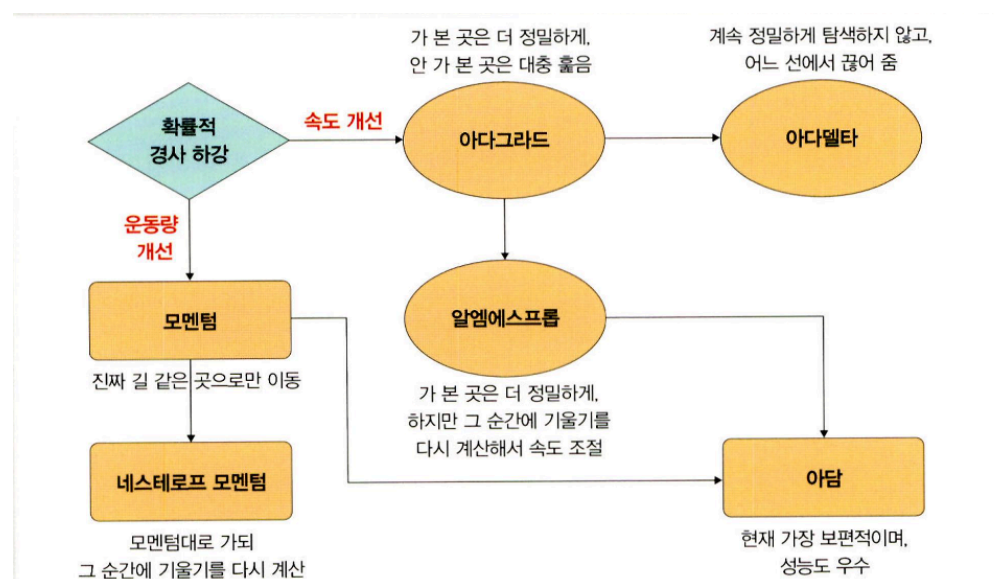
x = torch.FloatTensor(self.x_data[idx])
y = torch.FloatTensor(self.y_data[idx])
return x, y

dataset = CustomDataset()
dataloader = DataLoader(
    dataset, # 데이터셋
    batch_size = 2, # 미니 배치 크기로 2의 제곱수를 사용
    shuffle = True # 데이터를 불러올 때마다 랜덤으로 섞어서 가져옴
)

```

✓ 옵티마이저

- 확률적 경사 하강법의 파라미터 변경 폭이 불안정한 문제를 해결하기 위해 학습 속도와 운동량을 조정하는 옵티마이저를 적용해볼 수 있음



속도를 조정하는 방법

1. 아다그라드(Adagrad, Adaptive gradient)

- 변수(가중치)의 업데이트 횟수에 따라 학습률을 조정하는 방법
- 많이 변화하지 않는 변수들의 학습률을 크게 하고, 많이 변화하는 변수들의 학습률은 작게 함
 - 많이 변화한 변수는 최적 값에 근접했을 것이라는 가정하에 작은 크기로 이동하면서 세밀하게 값을 조정
 - 적게 변화한 변수들은 학습률을 크게 하여 빠르게 오차 값을 줄이고자 하는 방법
- 수식

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$

$$G(i) = G(i-1) + (\nabla E(w(i)))^2$$

- 파라미터마다 다른 학습률을 주기 위해 G 함수 추가
- G 값: 이전 G 값의 누적(기울기 크기의 누적)
- 기울기가 크면 G 값이 커지기 때문에 학습률은 작아짐
 - ⇒ 파라미터가 많이 학습되었으면 작은 학습률로 업데이트, 파라미터 학습이 덜 되었으면 개선의 여지가 많기 때문에 높은 학습률로 업데이트
- 기울기가 0에 수렴하는 문제가 있어 알엠에스프롬 사용
- 파이토치에서의 구현

```

optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01)
# 학습률 기본값은 1e-2

```

2. 아다델타(Adadelata, Adaptive delta)

- 아다그라드에서 G 값이 커짐에 따라 학습이 멈추는 문제를 해결하기 위해 등장한 방법
- 수식

$$w(i+1) = w(i) - \frac{\sqrt{D(i-1)+\varepsilon}}{\sqrt{G(i)+\varepsilon}} \nabla E(w(i))$$
$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$
$$D(i) = \gamma D(i-1) + (1-\gamma)(\Delta(w(i)))^2$$

- 학습률을 D함수(가중치 변화량 크기를 누적한 값)로 변환 ⇒ 학습률에 대한 하이퍼파라미터가 필요하지 않음

- 파이토치에서의 구현

```
optimizer = torch.optim.Adadelta(model.parameters(), lr=1.0)
# 학습률 기본값은 1.0
```

3. 알엠에스프롭(RMSProp)

- 아다그라드의 G(i) 값이 무한히 커지는 것을 방지하고자 제안된 방법
- 수식

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\varepsilon}} \nabla E(w(i))$$
$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

- 아다그라드에서 학습이 안되는 문제를 해결하기 위해 G 함수에서 감마만 추가
⇒ G 값이 너무 크면 학습률이 작아져 학습이 안 될 수 있으므로 사용자가 감마 값을 이용하여 학습률 크기를 비율로 조정할 수 있도록 함

- 파이토치에서의 구현

```
optimizer = torch.optim.RMSprop(model.parameters(), lr=0.01)
# 학습률 기본값은 1e-2
```

운동량을 조정하는 방법

1. 모멘텀(Momentum)

- 경사 하강법과 마찬가지로 매번 기울기를 구함
- 가중치를 수정하기 전에 이전 수정 방향(+, -)을 참고하여 같은 방향으로 일정한 비율만 수정하는 방법입니다
- 수정이 양(+)의 방향과 음(-)의 방향으로 순차적으로 일어나는 지그재그 현상이 줄어들고, 이전 이동 값을 고려하여 일정 비율만큼 다음 값을 결정
⇒ 관성 효과를 얻을 수 있는 장점
- 모멘텀은 SGD(확률적 경사 하강법)와 함께 사용
- 수식

먼저 확률적 경사 하강법의 수식이 다음과 같다고 합시다.

$$w(i+1) = w(i) - \eta \nabla E(w(i))$$

이때 $\eta \nabla E(w(i))$ 수식을 사용하여 가중치를 계산하는데, 기울기 크기와 반대 방향만큼 가중치를 업데이트합니다. 즉, 기울기가 크면 아래쪽(-) 방향으로 업데이트합니다.

또한, SGD 모멘텀(SGD with Momentum)은 확률적 경사 하강법에서 기울기($\eta \nabla E(w(i))$)를 속도(v , velocity)로 대체하여 사용하는 방식으로, 이전 속도의 일정 부분을 반영합니다. 즉, 이전에 학습했던 속도와 현재 기울기를 반영해서 가중치를 구합니다.

$$\begin{aligned} w(i+1) &= w(i) - v(i) \\ v(i) &= \gamma v(i-1) + \eta \nabla E(w(i)) \end{aligned}$$

• 파이토치에서의 구현

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

- momentum 값은 0.9에서 시작하여 0.95, 0.99처럼 조금씩 증가시키면서 사용

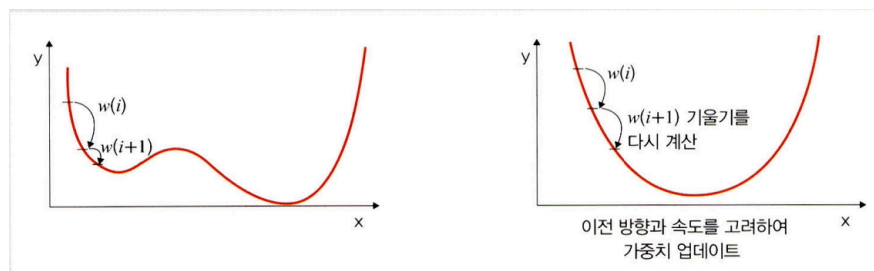
2. 네스테로프 모멘텀(Nesterov Accelerated Gradient, NAG)

모멘텀	네스테로프 모멘텀
기울기 값이 더해져 실제 값을 만듦	모멘텀 값이 적용된 지점에서 기울기 값을 계산
멈추어야 할 시점에서도 관성에 의해 훨씬 멀리 갈 수 있는 단점	모멘텀으로 절반 정도 이동한 후 어떤 방식으로 이동해야 하는지 다시 계산하여 결정 ⇒ 모멘텀의 단점 극복
	모멘텀의 이점인 빠른 이동 속도는 그대로 유지
	멈추어야 할 시점에서 제동을 거는 데 훨씬 용이

• 수식

$$\begin{aligned} w(i+1) &= w(i) - v(i) \\ v(i) &= \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1)) \end{aligned}$$

- 모멘텀과 비슷
- 속도(v)를 구하는 과정에서 차이가 있음
 - 이전에 학습했던 속도와 현재 기울기에서 이전 속도를 뺀 변화량을 반영해서 가중치를 구함



• 파이토치에서의 구현

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=True) # nestrev 기본값은 False
```

속도와 운동량에 대한 혼용 방법

1. 아담(Adam, Adaptive Moment Estimation)

- 모멘텀과 알엠에스프롭의 장점을 결합한 경사 하강법
- 수식
 - 알엠에스프롭 특징인 기울기의 제곱을 지수 평균한 값과 모멘텀 특징인 $v(i)$ 를 수식에 활용
⇒ 알엠에스프롭의 G 함수와 모멘텀의 $v(i)$ 를 사용하여 가중치를 업데이트

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\varepsilon}} v(i)$$

$$G(i) = \gamma_2 G(i-1) + (1-\gamma_2)(\nabla E(w(i)))^2$$

$$v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$$

- 파이토치에서의 구현

```
optimizer = torch.optim.Adam(mode.parameters(), lr=0.01) # 학습률 기본값은 1e-3
```

✓ 딥러닝을 사용할 때 이점

1. 특성 추출

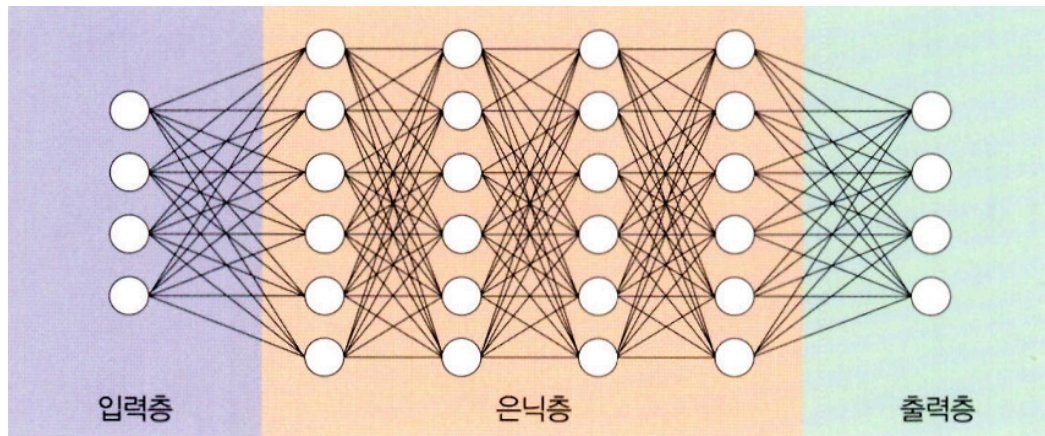
- 특성 추출: 데이터별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환하는 작업
- 딥러닝이 활성화되기 이전에 사용된 ML 알고리즘인 SVM, 나이브 베이즈, 로지스틱 회귀의 특성 추출은 매우 복잡하며 수집된 데이터에 대한 전문 지식이 필요했음
- 딥러닝에서는 이러한 특성 추출 과정을 알고리즘에 통합시킴
- 데이터 특성을 잘 잡아내고자 은닉층을 깊게 쌓는 방식으로 파라미터를 늘린 모델 구조를 가지기 때문

2. 빅데이터의 효율적 활용

- 빅데이터를 이용해 특성 추출을 알고리즘에 통합
- 딥러닝 학습을 이용한 특성 추출은 데이터 사례가 많을수록 성능이 향상됨
- 확보된 데이터가 적다면 딥러닝의 성능 향상을 기대하기 힘들기 때문에 머신러닝을 고려해 봐야함

4.3 딥러닝 알고리즘

1. 심층 신경망(DNN)



- 입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망
- 다수의 은닉층을 추가 ⇒ 머신러닝과 다르게 별도의 트릭 없이 비선형 분류가 가능

단점

- 학습을 위한 연산량이 많음
- 기울기 소멸 문제 등이 발생할 수 있음

⇒ 드롭아웃, 렐루 함수, 배치 정규화 등을 적용

2. 합성곱 신경망(Convolutional Neural Network, CNN)

- 합성곱층과 풀링층을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘

사용 분야

- 이미지 데이터(영상, 사진)에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용한 신경망
- 이미지에서 객체, 얼굴, 장면을 인식하기 위해 패턴을 찾는 데 유용

대표적인 CNN

- LeNet-5
- AlexNet

층을 더 깊게 쌓은 신경망

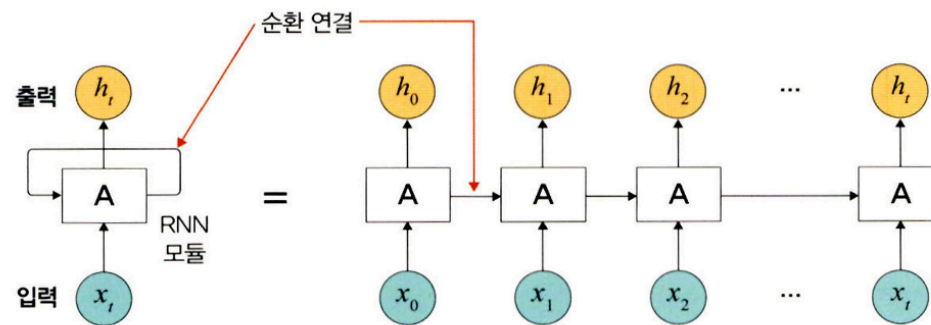
- VGG
- GoogLeNet
- ResNet

기존 신경망과의 차별성

- 각 층의 입출력 형상을 유지
- 이미지의 공간 정보를 유지하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식
- 복수 필터로 이미지의 특징을 추출하고 학습
- 추출한 이미지의 특징을 모으고 강화하는 풀링층이 있음
- 필터를 공유 파라미터로 사용 ⇒ 일반 인공 신경망과 비교하여 학습 파라미터가 매우 적음

3. 순환 신경망(RNN)

- 시계열 데이터(음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망
- 순환 신경망의 '순환(recurrent)'는 자기 자신을 참조한다는 뜻
⇒ 현재 결과가 이전 결과와 연관이 있다는 의미



특징

- 시간성을 가진 데이터가 많음
- 시간성 정보를 이용하여 데이터의 특징을 잘 다룸
- 시간에 따라 내용이 변하므로 데이터는 동적이고, 길이가 가변적
- 매우 긴 데이터를 처리하는 연구가 활발히 진행되고 있음

단점

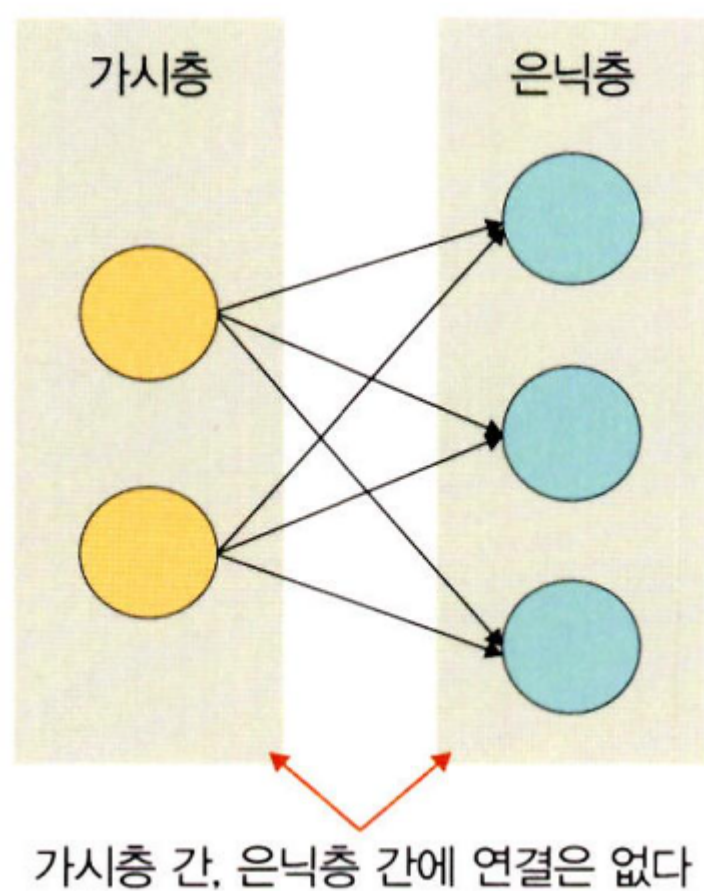
- 기울기 소멸 문제로 학습이 제대로 되지 않는 문제가 있음
⇒ 메모리 개념을 도입한 LSTM이 많이 사용됨

사용 분야

- 자연어 처리 분야와 잘 맞음
- 예) 언어 모델링, 텍스트 생성, 자동 번역(기계 번역), 음성 인식, 이미지 캡션 생성

4. 제한된 볼츠만 머신(RBM)

- 볼츠만 머신: 가시층과 은닉층으로만 구성된 모델
- 제한된 볼츠만 머신: 가시층은 은닉층과만 연결. 가시층과 가시층, 은닉층과 은닉층 사이에 연결이 없음



특징

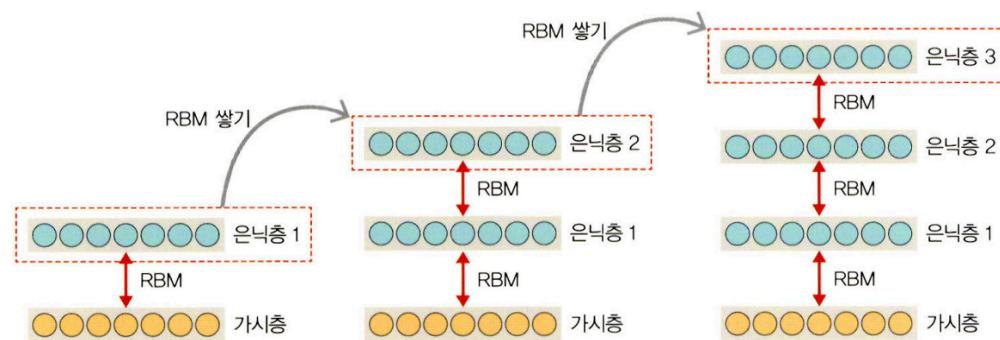
- 차원 감소, 분류, 선형 회귀 분석, 협업 필터링, 특성 값 학습, 주제 모델링에 사용
- 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용 가능
- 심층 신뢰 신경망(DBN)의 요소로 활용

- 딥러닝에서 많이 사용되는 알고리즘: CNN, RNN
- 제한된 볼츠만 머신, 심층 신뢰 신경망은 상대적으로 덜 사용

5. 심층 신뢰 신경망(Deep Belief Network, DBN)

- 사전 훈련된 제한된 볼츠만 머신을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망
- 레이블 없는 데이터에 대한 비지도 학습이 가능
- 부분적인 이미지에서 전체를 연상하는 일반화와 추상화 과정을 구현할 때 사용

학습 절차



1. 가시층과 은닉층1에 제한된 볼츠만 머신을 사전 훈련
2. 첫번째 층 입력 데이터와 파라미터를 고정하여 두번째 층 제한된 볼츠만 머신을 사전 훈련
3. 원하는 층 개수만큼 제한된 볼츠만 머신을 쌓아 올려 전체 DBN을 완성

특징

- 순차적으로 심층 신뢰 신경망을 학습시켜 가면서 계층적 구조를 생성
- 비지도 학습으로 학습
- 위로 올라갈수록 추상적 특성을 추출
- 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용